PHASE 1 — CORE AL ENGINEERING

W1D1 → W8D6

Project: University Student & Admissions Core System

Goal: Strong AL foundation, clean architecture, no bad patterns

---

WEEK 1 — FOUNDATION & DATA MODELING

W1D1 — Environment & Architecture Setup

Theory

- AL project structure
- app.json
- object ranges & naming standards

Build

- Create AL extension project
- Setup folders:
    - Tables
    - Pages
    - Codeunits
    - Enums
    - Interfaces
- Configure app.json properly

Output

- Clean scaffolded project pushed to repo

W1D2 — Core Data Modeling (Student Table I)

Theory

- Tables in BC
- Field types & primary keys

Build

Create Student Table:

- Student No
- Full Name
- National ID
- Email
- Phone
- Status
- Created Date

Output

- Persistent Student entity

W1D3 — Table Logic & Validation Layer

Theory

- Table triggers vs codeunit logic
- Data validation principles

Build

- Auto-numbering (No Series concept simulation)

- Email validation
- Mandatory field enforcement

Output

- Validated student creation flow

W1D4 — UI Layer (Pages)

Theory

- Card vs List pages
- Page actions

Build

- Student List Page
- Student Card Page
- Actions:
    - New Student
    - Edit Student

Output

- Fully usable UI

W1D5 — Business Logic Separation (Codeunits)

Theory

- Why logic must NOT live in tables/pages

Build

Create:

- Student Management Codeunit

Move logic:

- CreateStudent()
- ValidateStudent()
- UpdateStudent()

Output

- Clean separation of concerns

W1D6 — WEEK 1 TEST (GATE 1)

TEST TASKS

- Create student via UI
- Validate email rules
- Edit student record
- Confirm numbering works
- Break validation intentionally

PASS CRITERIA

- No logic in table/page layers
- Code clean and readable
- Flow works end-to-end

WEEK 2 — EVENTS & CLEAN ARCHITECTURE

W2D1 — Event Fundamentals

Theory

- Publisher vs subscriber pattern

Build

- Add integration event:
    - OnBeforeStudentCreated
    - OnAfterStudentCreated

W2D2 — Event Subscribers

Theory

- Decoupling logic

Build

- Subscriber Codeunit:
    - Send welcome notification (log-based for now)

W2D3 — Business Decoupling

Theory

- Why tight coupling breaks systems

Build

- Move notification logic OUT of core codeunit
- Replace direct calls with events

W2D4 — Event Expansion Pattern

Theory

- Extensibility mindset

Build

- Add:
    - OnStudentStatusChanged event
- Hook subscriber logic

W2D5 — Refactor Day

Build

- Refactor all Week 2 logic
- Remove direct dependencies

Output

- Fully event-driven student module

W2D6 — WEEK 2 TEST (GATE 2)

TEST TASKS

- Create student
- Confirm event triggers fire
- Confirm subscriber runs
- Remove subscriber → system still works

PASS CRITERIA

- Zero tight coupling
- Event flow works correctly

WEEK 3 — ENUMS & EXTENSIBILITY

W3D1 — Replace Option Fields

Theory

- Why enums replace options in modern BC

Build

- Convert Status field → Enum

W3D2 — Enum Extensions

Build

- Extend Student Status enum:
    - Active
    - Suspended
    - Graduated

W3D3 — Enum Logic Refactor

Build

- Replace all IF/CASE status logic with enum patterns

W3D4 — Clean Architecture Check

Theory

- Avoid hardcoding values

Build

- Remove all hardcoded statuses

W3D5 — Reusability Layer

Build

- Create Status Helper Codeunit

W3D6 — WEEK 3 TEST (GATE 3)

TEST TASKS

- Change status behavior
- Extend enum
- Ensure system still works

PASS CRITERIA

- No hardcoded logic remains

WEEK 4 — API FOUNDATION (READ MODEL)

W4D1 — API Page Creation

Build

- Create Student API Page
- Expose fields properly

W4D2 — API Testing Setup

Build

- Use Postman
- Test GET requests

W4D3 — CRUD via API (Read/Write)

Build

- Enable POST/PUT operations

W4D4 — Data Mapping Rules

Theory

- DTO mapping concept

Build

- Create mapping logic codeunit

W4D5 — Error Handling API Layer

Build

- Handle invalid payloads

W4D6 — WEEK 4 TEST (GATE 4)

TEST TASKS

- Create student via API
- Update via API
- Fetch via API

PASS CRITERIA

- API fully functional

WEEK 5 — ERROR HANDLING & LOGGING FRAMEWORK

W5D1 — Logging Table Design

Build

- Integration Log Table

W5D2 — Logging Codeunit

Build

- WriteLog()
- CaptureRequest()
- CaptureResponse()

W5D3 — Error Handling Patterns

Theory

- TryFunction
- Controlled failures

W5D4 — Central Error Handler

Build

- Global error handler codeunit

W5D5 — Logging Integration

Build

- Attach logs to student actions

W5D6 — WEEK 5 TEST (GATE 5)

TEST TASKS

- Force API failure
- Validate logs captured
- Validate system stability

WEEK 6 — REFACTOR & CONSOLIDATION

W6D1 — Architecture Review

- Identify bad design

W6D2 — Refactor tables/pages

W6D3 — Refactor codeunits

W6D4 — Remove duplication

W6D5 — Optimize structure

W6D6 — WEEK 6 FINAL TEST (PHASE GATE 1 CHECKPOINT)

FULL SYSTEM TEST

- Student creation
- API flow
- Events
- Logging
- Enum system

PASS CRITERIA

- Clean architecture
- No tight coupling

- Stable system

WEEK 7 — MINI INTEGRATION INTRO

W7D1

- HttpClient basics

W7D2

- External API GET

W7D3

- JSON parsing

W7D4

- Store external data

W7D5

- Error handling external calls

W7D6 — TEST

- Consume public API successfully

WEEK 8 — PHASE 1 FINAL SYSTEM

W8D1

- Begin full system consolidation

W8D2

- Add integration touchpoints

W8D3

- Improve UI + UX logic

W8D4

- Final refactor

W8D5

- Full system testing

W8D6 — FINAL PHASE 1 TEST (EXIT GATE)

FINAL TESTS

- Student lifecycle works end-to-end
- API works
- Events fire correctly
- Logging works
- Code is clean and reusable

PHASE 1 EXIT RULE

You ONLY move to Phase 2 if:

- System works without hacks
- Code is clean and modular
- You can explain architecture confidently
- You pass W8D6 full test